

Software Requirements Specification (SRS)

Snake Game

Project: Snake Game (Console and Graphics Interface)

Version: 1.0

Author: Pradyumna

SRN: PES2UG24CS359

Course: Software Engineering – Mini Project

Date: 06-09-2026

Status:

Project: Snake Game (Console and Graphics Interface)

Version: 1.0

Author: Noel

SRN: PES2UG24CS330

Course: Software Engineering – Mini Project

Date: 06-09-2026

Status:

Project: Snake Game (Console and Graphics Interface)

Version: 1.0

Author: Sudeshna

SRN: PES2UG24CS313

Course: Software Engineering – Mini Project

Date: 06-09-2026

Status:

Project: Snake Game (Console and Graphics Interface)

Version: 1.0

Author: Shyam

SRN: PES2UG24CS349

Course: Software Engineering – Mini Project

Date: 06-09-2026

Status:

Revision History

Version	Date	Author	Change summary	Approval

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			

Table of Contents

- 1. Introduction
- 2. Overall Description
- 3. External Interface Requirements
- 4. System Features (Detailed)
- 5. Non-Functional Requirements
- 6. Quality Attributes & Acceptance Tests
- 7. System Models and Diagrams
- 8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This document is a Software Requirements Specification (SRS) for the Snake Game mini project developed as part of the Software Engineering course. It defines the functional and non-functional requirements, external interfaces, and verification criteria for a single-player Snake game that supports both a console-based (text/ASCII) interface and a graphics-based interface, implemented in C/C++.

1.2 Scope

The Snake Game is a standalone, offline, single-player desktop application in which the player controls a snake that moves around a bounded playing field, consumes food items to grow longer, and must avoid colliding with the boundary walls, static obstacles, and its own body. The system covers gameplay logic, both console and graphical rendering modes, input handling, collision detection, and session-based score tracking. The scope excludes networked/multiplayer play, persistent (cross-session) high-score storage, and any mobile platform builds.

1.3 Audience

Developers (students), Course Instructor, Course Evaluator, Testers, players, System Integrators

1.4 Definitions, Acronyms and Abbreviations

- SRS – Software Requirements Specification
- FR – Functional Requirement
- NFR – Non-Functional Requirement
- CLI – Command Line Interface (console mode)
- GUI – Graphical User Interface (graphics mode)
- FPS – Frames Per Second
- ASCII – American Standard Code for Information Interchange
- RTM – Requirements Traceability Matrix

2. Overall Description

2.1 Product Perspective

The Snake Game is a new, self-contained desktop application with no dependency on any external banking, web, or cloud system. It is built in C/C++ and provides two selectable rendering front-ends: a console/text-based mode and a graphics-based mode, both driven by the same core game-logic engine. It receives player commands through the keyboard, updates the game state at regular intervals, checks food and collisions, and displays the resulting state. A local file may be used to preserve the high score.

2.2 Major Product Functions

- Interface mode selection (console or graphics).
- Start a new game and initialize the score, food and Snake.
- Snake movement and directional control via keyboard input
- Random food generation and snake growth on consumption
- Static obstacle/wall placement and collision detection
- Self-collision (snake hitting its own body) detection
- Real-time, session-based score tracking and display
- Game-over detection, on-screen result display, and restart
- Pause and resume of an in-progress game session

2.3 User Roles and Characteristics

- Player: general user with no special training; expects immediate, responsive controls and clear visual/text feedback.
- Developer/Tester: builds, runs, and verifies both interface modes across target platforms during development and grading.

2.4 Operating Environment

The target environment is a standard desktop or laptop computer running Windows, Linux or macOS. A graphical version requires a supported graphics/game library; a console version requires a terminal or command prompt. A keyboard and display are required.

2.5 Design and Implementation Constraints

- The application shall be implemented in C/C++, using only libraries and tools available in the target laboratory environment.
- The game shall support keyboard-based controls, with configurable board dimensions and movement speed.
- A fixed game-loop tick rate shall be used to ensure consistent movement and gameplay across machines with different performance levels.
- The application shall run locally without internet or network access, with no dependency on external databases.
- All game state and scores shall be maintained in memory for the duration of a session.
- The implementation shall remain reasonably modular by separating game logic, input handling, display/rendering, and score management, while maintaining unique requirement IDs for traceability.

3. External Interface Requirements

3.1 User Interfaces

Console mode: a fixed-size character grid representing the playing field, with the snake, food, obstacles, and score rendered using text/ASCII characters, refreshed on a fixed timer. Graphics mode: a resizable/fixed window with the same playing field rendered as coloured shapes/sprites, plus an on-screen score display and a start/game-over menu screen.

3.2 Hardware Interfaces

- Keyboard: arrow keys or W/A/S/D for directional control; Enter/Space to start, pause, and restart; Esc/Q to quit.
- Display monitor for rendering the console window or graphics window.

3.3 Software Interfaces

- Console I/O library for non-blocking keyboard input and cursor positioning.
- Game Logic Module: Takes the current game state and direction command as input and updates movement, food, and collision states.
- Graphics library for window creation, drawing primitives, and event handling in graphics mode.
- Operating system timer/clock functions for fixed-interval game-loop ticking.
- High Score Storage: Takes the current score and stores/retrieves the highest score for local preservation.

3.4 Communications Interfaces

Not applicable. The Snake Game is a standalone, offline application and does not communicate over any network.

4. System Features (Detailed)

Each requirement below includes its acceptance criteria and a reference test case. Requirement IDs follow the pattern SNK-F-###.

4.1 Interface Mode Selection

Description: Allow the player to choose between console-based and graphics-based rendering before a game session starts.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SNK-F-001	The system shall present a startup menu allowing the player to choose console mode or graphics mode.	Functional	High	Player	AC-SNK-F-001: Menu is shown before gameplay begins and accepts a valid selection. Test: TC-Menu-01	
SNK-F-002	The system shall initialize the chosen rendering mode (console or graphics) without requiring a recompile.	Functional	High	Developer	AC-SNK-F-002: Selected mode loads correctly on first attempt. Test: TC-Menu-02	Depends on library availability on target OS
SNK-F-003	The system shall reject invalid menu input and re-prompt the player instead of crashing.	Functional	Medium	Player	AC-SNK-F-003: Invalid key press re-displays menu, no crash. Test: TC-Menu-03	

4.2 Snake Movement and Control

Description: Move the snake continuously in the current direction and let the player change direction using keyboard input.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SNK-F-004	The system shall move the snake one grid cell forward, in its current direction, at every game-loop tick.	Functional	High	Player	AC-SNK-F-004: Snake advances exactly one cell per tick at the configured speed. Test: TC-Move-01	
SNK-F-005	The system shall change the snake's direction of travel based on arrow-key/WASD input, taking effect on the next tick.	Functional	High	Player	AC-SNK-F-005: Direction changes match key pressed within one tick. Test: TC-Move-02	
SNK-F-006	The system shall prevent the snake from reversing directly into itself (e.g., moving Right while currently moving Left).	Functional	High	Player	AC-SNK-F-006: Opposite-direction key press while moving is ignored. Test: TC-Move-03	
SNK-F-007	The system shall keep the snake's speed constant within a session unless a difficulty change is explicitly implemented.	Functional	Medium	Player	AC-SNK-F-007: Tick interval remains constant through a session. Test: TC-Move-04	

4.3 Food Generation and Growth

Description: Place food items at random empty cells and grow the snake when food is eaten.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SNK-F-008	The system shall randomly place one food item on an empty grid cell not occupied by the snake or an obstacle.	Functional	High	Player	AC-SNK-F-008: Food never spawns on the snake body or an obstacle. Test: TC-Food-01	
SNK-F-009	The system shall increase the snake's length by one segment when the snake's head enters the food's cell.	Functional	High	Player	AC-SNK-F-009: Snake length increases by exactly one segment per food eaten. Test: TC-Food-02	
SNK-F-010	The system shall generate a new food item immediately after the current one is consumed.	Functional	High	Player	AC-SNK-F-010: A new food cell appears within the same tick/frame after consumption. Test: TC-Food-03	Depends on SNK-F-008

4.4 Obstacle and Boundary Collision

Description: Detect and respond to the snake colliding with the outer wall boundary or a static obstacle.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SNK-F-011	The system shall define a fixed set of static obstacle cells on the playing field at the start of each session.	Functional	High	Player	AC-SNK-F-011: Obstacles are drawn/rendered at fixed positions before gameplay starts. Test: TC-Obs-01	
SNK-F-012	The system shall detect when the snake's head moves into a boundary wall cell and end the game.	Functional	High	Player	AC-SNK-F-012: Game-over triggers immediately on wall contact. Test: TC-Obs-02	
SNK-F-013	The system shall detect when the snake's head moves into an obstacle cell and end the game.	Functional	High	Player	AC-SNK-F-013: Game-over triggers immediately on obstacle contact. Test: TC-Obs-03	Depends on SNK-F-011

4.5 Self-Collision and Score Tracking

Description: Detect collision with the snake's own body and maintain the session score.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SNK-F-014	The system shall detect when the snake's head moves into any cell occupied by its own body and end the game.	Functional	High	Player	AC-SNK-F-014: Game-over triggers on self-collision. Test: TC-Score-01	
SNK-F-015	The system shall increase the player's session score by a fixed amount each time food is consumed.	Functional	High	Player	AC-SNK-F-015: Score increments correctly and is visible immediately. Test: TC-Score-02	Depends on SNK-F-009
SNK-F-016	The system shall continuously display the current score on-screen throughout gameplay, in both console and graphics modes.	Functional	Medium	Player	AC-SNK-F-016: Score value on screen matches internal score at all times. Test: TC-Score-03	

4.6 Game Over, Restart and Session Control

Description: Handle end-of-game display, restart, pause/resume, and exit.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SNK-F-017	The system shall display a game-over screen showing the final score when the game ends.	Functional	High	Player	AC-SNK-F-017: Final score shown matches last recorded score. Test: TC-Session-01	
SNK-F-018	The system shall allow the player to restart a new game session from the game-over screen without restarting the application.	Functional	High	Player	AC-SNK-F-018: Restart resets snake, score, food, and obstacles to initial state. Test: TC-Session-02	
SNK-F-019	The system shall allow the player to pause and resume an in-progress game session.	Functional	Medium	Player	AC-SNK-F-019: Pause halts snake movement; resume continues from the same state. Test: TC-Session-03	
SNK-F-020	The system shall allow the player to exit the game cleanly from the menu or during gameplay.	Functional	Medium	Player	AC-SNK-F-020: Application terminates without an error or crash. Test: TC-Session-04	

5. Non-Functional Requirements (Detailed)

NFRs below are measurable and tied to test plans. Requirement IDs follow the pattern SNK-NF-####.

Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
SNK-NF-001	The game loop shall process one full tick (input, update, render) within 50 ms to sustain smooth movement at the configured speed.	Performance	High	Tick processing time measured ≤ 50 ms in 95% of ticks during a test session. Test: TC-Perf-01
SNK-NF-002	The application shall not crash or hang during a continuous 30-minute play session under normal input.	Reliability	High	No crash/hang observed across 5 consecutive 30-minute soak-test runs. Test: TC-Rel-01
SNK-NF-003	Both console and graphics modes shall build and run without modification on both Windows and Linux target machines.	Portability	Medium	Successful build and run verified on both target OS platforms. Test: TC-Port-01
SNK-NF-004	Game controls shall be simple enough that a first-time player can start playing correctly within 30 seconds of the instructions shown.	Usability	Medium	Usability walkthrough with a new user completes first correct move within 30 seconds. Test: TC-UX-01
SNK-NF-005	The core game-logic module shall be implemented independently of the rendering mode so a new interface can be added without changing game logic.	Maintainability	Medium	Code review confirms game-logic functions have no direct console/graphics library calls. Test: TC-Maint-01

5.1 Security

5.1.1 Security Objectives

- Prevent memory-safety defects (e.g., buffer overflows, out-of-bounds array access) in the C/C++ implementation that could crash the application or corrupt game state.
- Ensure the application only ever acts on well-formed, expected keyboard input and cannot be driven into an undefined or unstable state by unexpected input.

5.1.2 Security Requirements

Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / Test case ref
SNK-SR-001	The system shall validate that all array indices used for the snake body, food position, and obstacle grid remain within the bounds of the playing field at all times.	Security	High	AC-SNK-SR-001: No out-of-bounds index reachable in code review/static analysis. Test: TC-Sec-01
SNK-SR-002	The system shall use bounded/safe input-buffer functions (avoiding unsafe functions such as gets()) when reading any text input.	Security	High	AC-SNK-SR-002: Static analysis shows no use of unsafe unbounded input functions. Test: TC-Sec-02
SNK-SR-003	The system shall ignore and discard any keyboard input not mapped to a recognized game control, without altering game state.	Security	Medium	AC-SNK-SR-003: Unmapped key presses produce no state change. Test: TC-Sec-03
SNK-SR-004	The system shall not allocate memory dynamically inside the per-tick game loop in a way that can grow unbounded (e.g., snake segment list capped to grid size).	Security	Medium	AC-SNK-SR-004: Memory usage remains bounded and stable across an extended play session. Test: TC-Sec-04
SNK-SR-005	The system shall run entirely offline with no listening network sockets or external inputs, eliminating remote-attack surface.	Security	High	AC-SNK-SR-005: Network monitoring during execution shows no open sockets. Test: TC-Sec-05

6. Quality Attributes & Acceptance Tests

Exit criteria for acceptance: all High-priority functional requirements are implemented and verified, no High-priority NFR or security requirement fails, and the RTM shows all associated test cases passed.

Acceptance test suites: Interface Mode Selection, Movement & Control, Food & Growth, Collision (wall/obstacle/self), Score & Session Control, Performance, Reliability, Portability, and Security.

7. System Models and Diagrams

7.1 UML Use-Case Diagrams

The following two use-case diagrams model the Player's interactions with the Snake Game system: core gameplay actions, and collision/game-over/session-control actions.

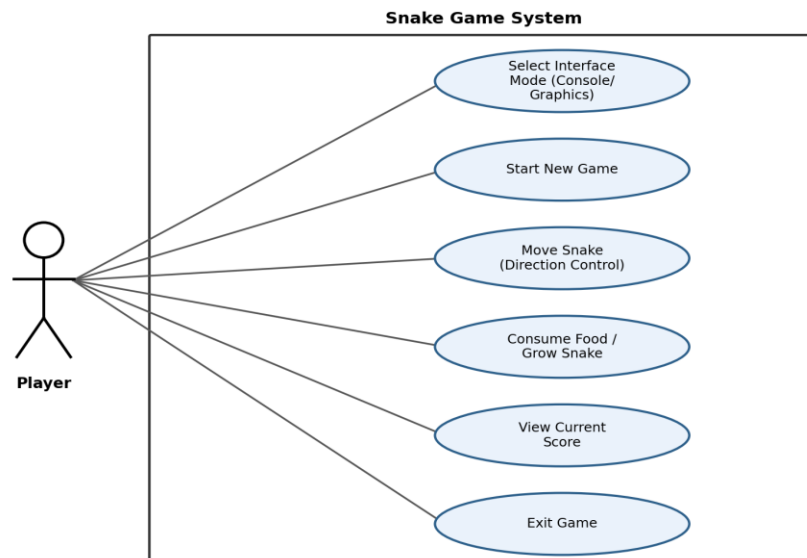


Figure 7.1: Use-Case Diagram – Core Gameplay Interaction

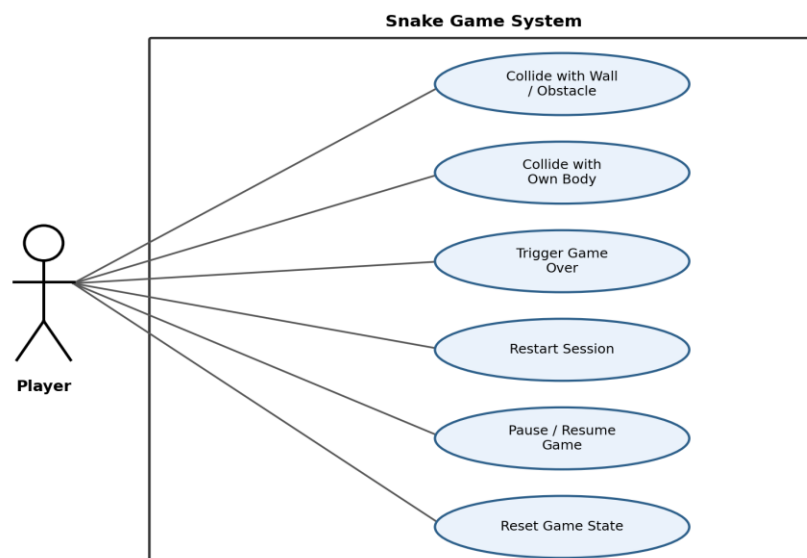


Figure 7.2: Use-Case Diagram – Collision, Game-Over & Session Control

8. Requirements Traceability Matrix (RTM)

Req ID	Requirement (short)	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
SNK-F-001	Interface mode menu	4.1 / DS-Menu-01	MenuModule	TC-Menu-01	N	
SNK-F-004	Snake movement per tick	4.2 / DS-Move-01	GameLogicModule	TC-Move-01	N	
SNK-F-006	Prevent reverse direction	4.2 / DS-Move-01	GameLogicModule	TC-Move-03	N	
SNK-F-008	Random food placement	4.3 / DS-Food-01	FoodModule	TC-Food-01	N	
SNK-F-009	Growth on food eaten	4.3 / DS-Food-01	GameLogicModule	TC-Food-02	N	
SNK-F-012	Wall collision → game over	4.4 / DS-Collision-01	CollisionModule	TC-Obs-02	N	
SNK-F-013	Obstacle collision → game over	4.4 / DS-Collision-01	CollisionModule	TC-Obs-03	N	
SNK-F-014	Self-collision → game over	4.5 / DS-Collision-01	CollisionModule	TC-Score-01	N	
SNK-F-015	Score increment	4.5 / DS-Score-01	ScoreModule	TC-Score-02	N	
SNK-F-018	Restart session	4.6 / DS-Session-01	SessionModule	TC-Session-02	N	
SNK-NF-001	Tick latency ≤ 50ms	5 / DS-Perf-01	GameLoop	TC-Perf-01	N	
SNK-NF-002	30-min stability	5 / DS-Rel-01	GameLoop	TC-Rel-01	N	
SNK-SR-001	Bounds-checked indices	5.1.2 / DS-Sec-01	GameLogicModule	TC-Sec-01	N	
SNK-SR-002	Safe input functions	5.1.2 / DS-Sec-01	InputModule	TC-Sec-02	N	
SNK-SR-005	No network exposure	5.1.2 / DS-Sec-01	Application	TC-Sec-05	N	